



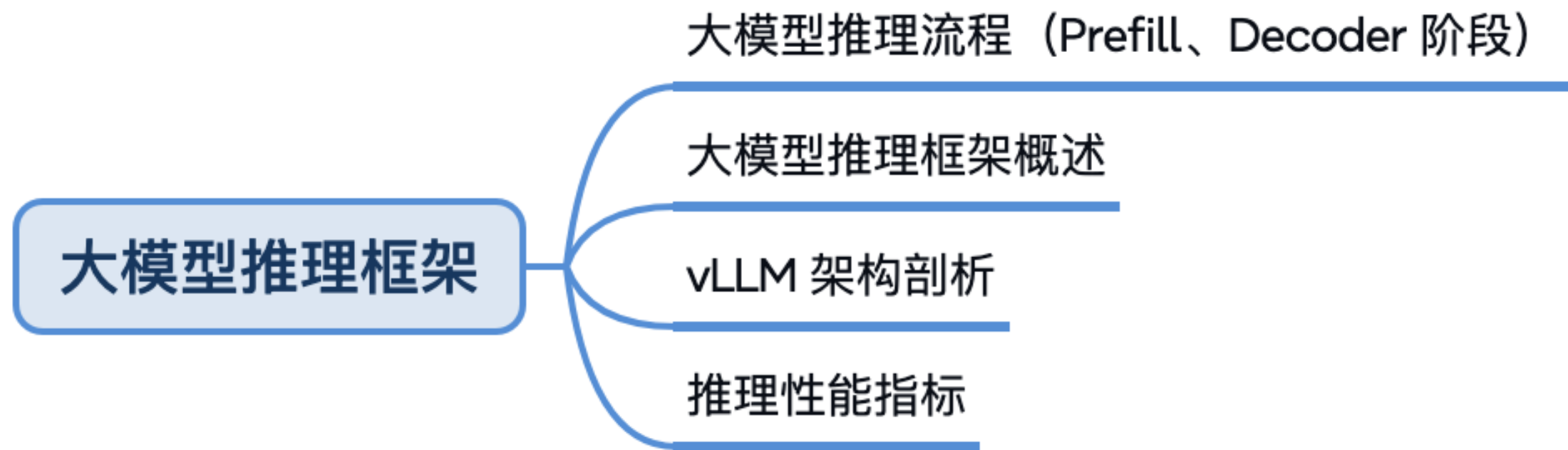
vLLM 架构剖析



ZOMI



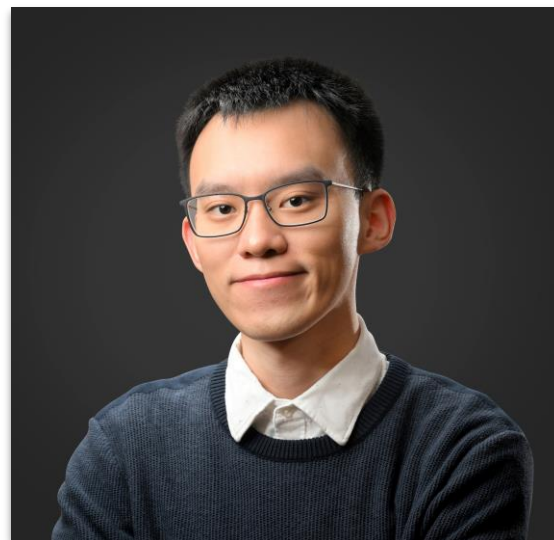
大模型推理



About us



Woosuk Kwon
PhD Student, UC Berkeley
 [@woosuk_k](#)



Zhuohan Li
PhD Student, UC Berkeley
 [@zhuohan123](#)



目录

1. vLLM Overview
2. vLLM 调用方式
3. vLLM 整体代码架构
4. vLLM 加载模型 & 预分配显存
5. vLLM Scheduler 调度
6. vLLM 整体架构总结



01

vLLM Overview



Our Goal

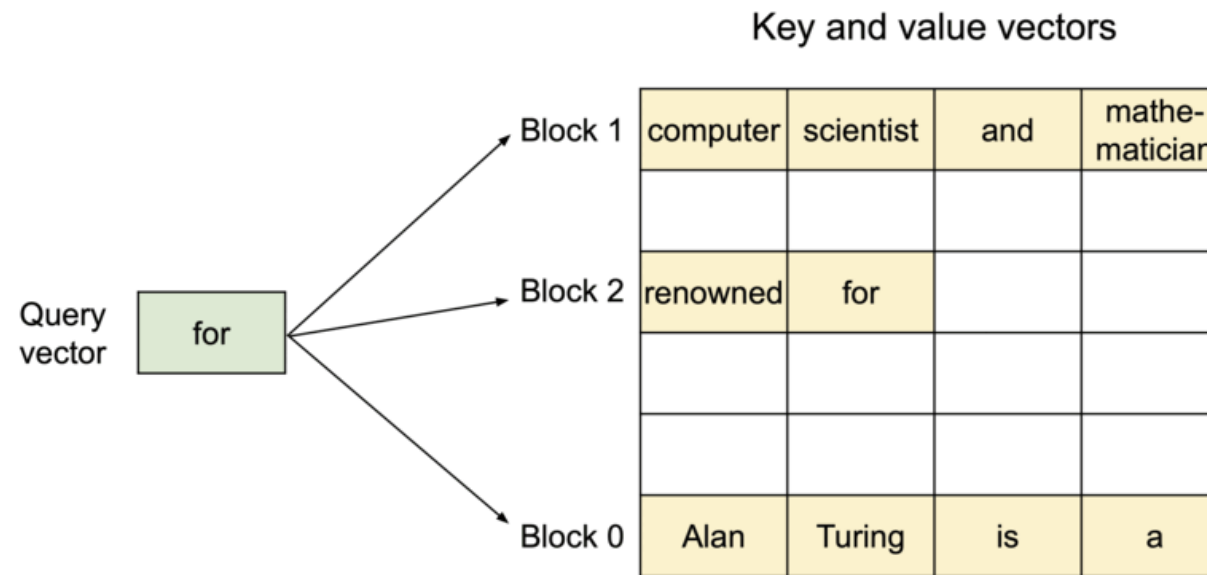
结合 iterative-level schedule (continuous batching) 和 PagedAttention 注意力算法以提高服务的吞吐量。

Build the **fastest** and **easiest-to-use** open-source LLM inference & serving engine

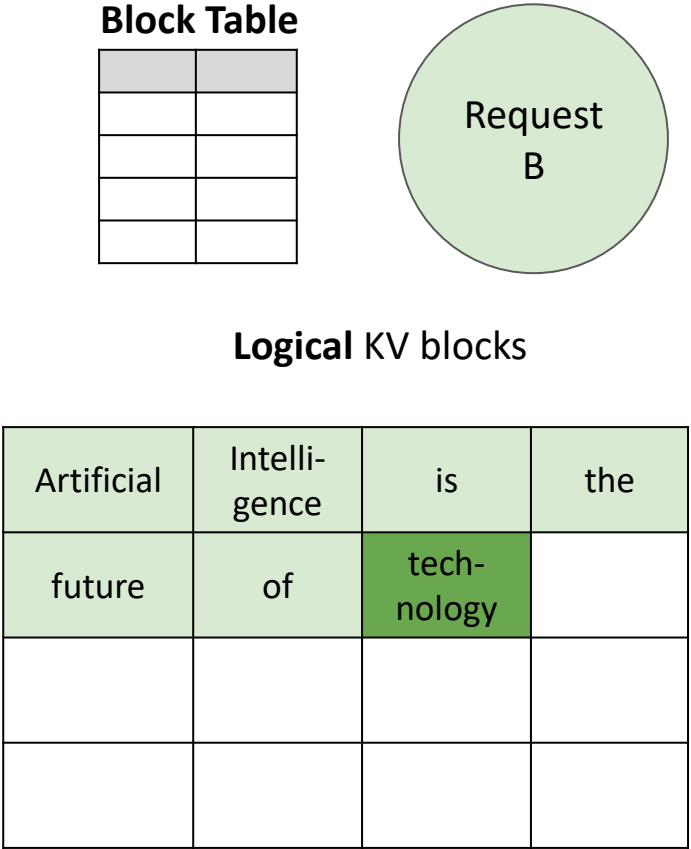
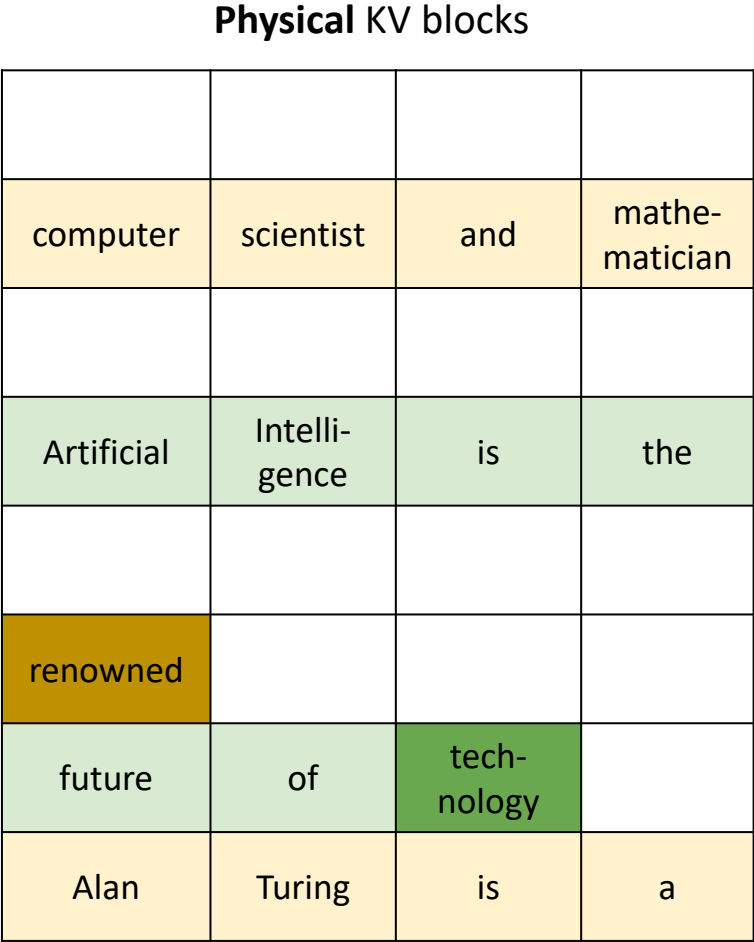
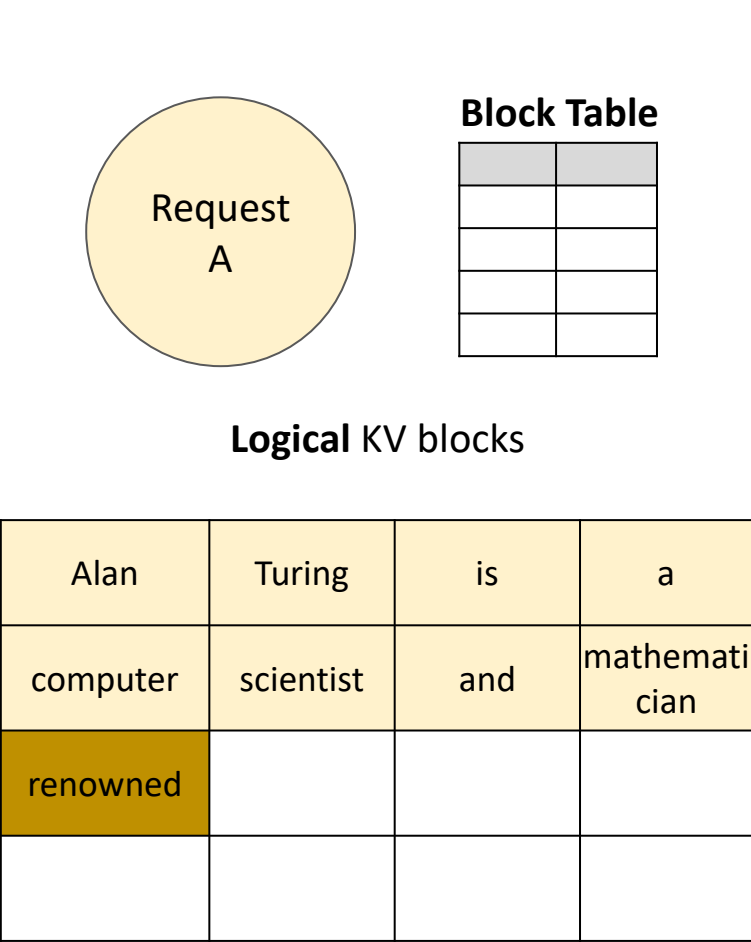


PagedAttention

An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space.



Manage KV cache like OS virtual memory



vLLM Github Repo



<https://github.com/vllm-project/vllm>

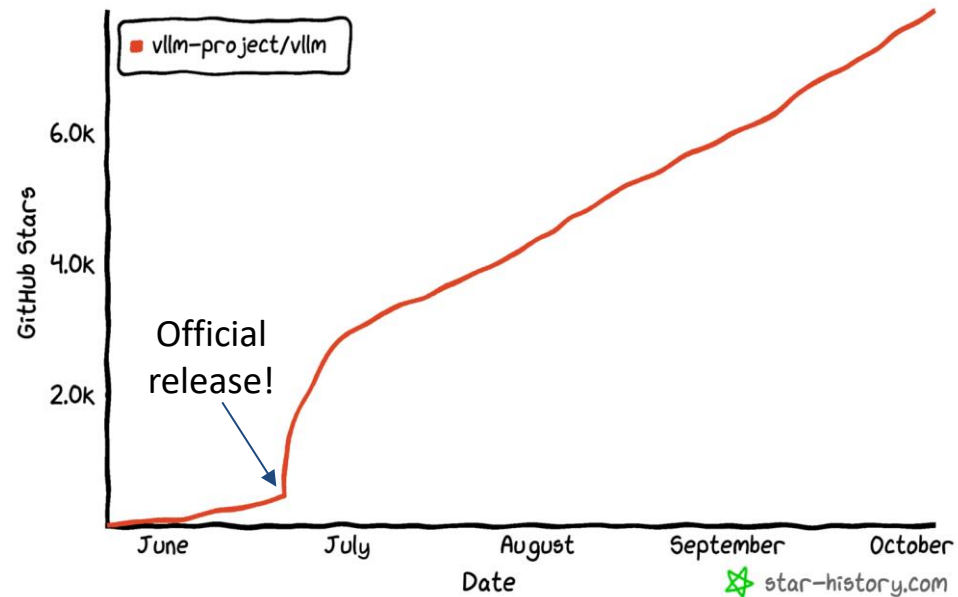


`$ pip install vllm`



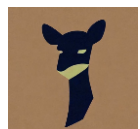
33.9K Stars

Star History



LLM Adopters

Open-Source Projects



Im-sys/FastChat



OPENCHAT

Ai2 allenai/open-instruct



SkyPilot



WizardLM

...

Companies



anyscale



databricks



Meta



Replicate



Google Cloud



Modal

scale

...



BentoML



Lepton AI



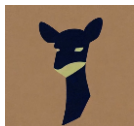
NVIDIA

Uber



ZOMI

“Talk is cheap. Show me the code.”



Im-sys/FastChat

https://github.com/lm-sys/FastChat/blob/main/fastchat/serve/vllm_worker.py



https://github.com/imoneoi/openchat/blob/master/ochat/evaluation/run_eval.py



https://github.com/GoogleCloudPlatform/vertex-ai-samples/tree/main/community-content/vertex_vision_model_garden/model_oss/vllm



<https://github.com/leptonai/examples/tree/main/advanced/vllm>



<https://github.com/facebookresearch/llama-recipes/tree/main/examples/vllm>



https://github.com/triton-inference-server/tutorials/tree/main/Quick_Deploy/vLLM



https://github.com/modal-labs/modal-examples/blob/main/06_gpu_and_ml/vllm_inference.py



<https://github.com/replicate/vllm-with-loras>

https://github.com/scaleapi/llm-engine/tree/main/model-engine/model_engine_server/inference/vllm



<https://github.com/skypilot-org/skypilot/blob/master/llm/vllm/serve.yaml>



WizardLM

https://github.com/nlpxucan/WizardLM/blob/main/WizardCoder/src/humaneval_gen_vllm.py



https://github.com/ray-project/ray/blob/master/doc/source/serve/doc_code/vllm_example.py



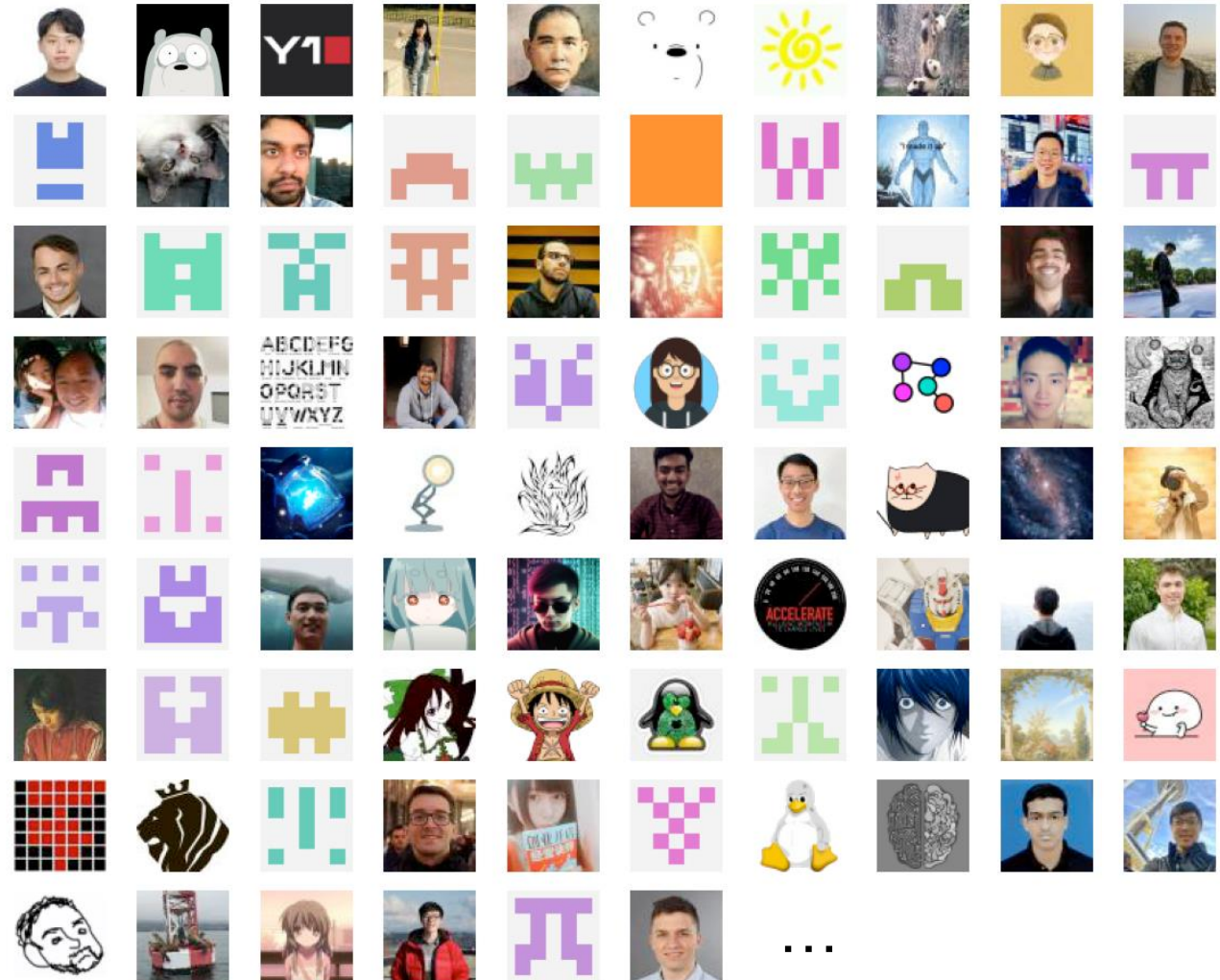
https://github.com/bentoml/OpenLLM/blob/main/openllm-python/src/openllm/models/auto/modeling_vllm_auto.py



ZOMI

Contributors

Thanks to all the contributors who raised issues, participated in discussions, and submitted PRs!



02

VLLM

调用方式

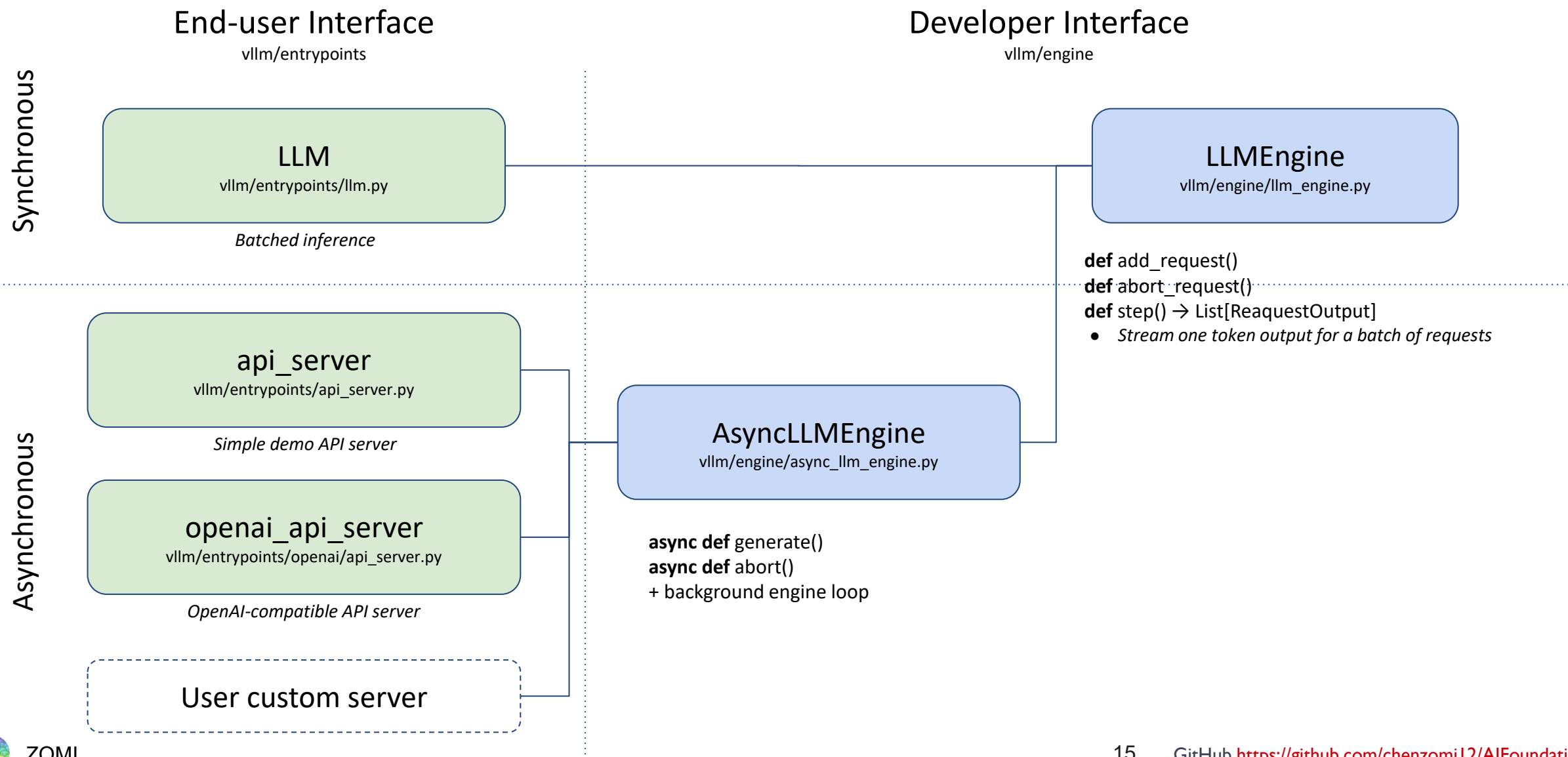


vLLM 推理调用方式

- 根据 vLLM 官方文档，向用户提供了两种调用它的方法，分别是：
 - Offline Batched Inference（同步，离线批处理）
 - API Server For Online Serving（异步，在线推理服务）



User Interface



vLLM API (1): LLM class

A Python interface for offline batched inference

```
from vllm import LLM

# Example prompts.
prompts = ["Hello, my name is", "The capital of France is"]
# Create an LLM with HF model name.
llm = LLM(model="meta-llama/Llama-2-7b-hf")
# Generate texts from the prompts.
outputs = llm.generate(prompts)
```

离线批处理

- 在传统离线批处理中，我们每次给模型发送推理请求时，都要：
 1. 等一个batch的数据齐全后，一起发送
 2. 整个batch的数据一起做推理
 3. 等一个batch的数据全部推理完毕后，一起返回推理结果



vLLM API (2): OpenAI-compatible server

A FastAPI-based server for online serving

Server

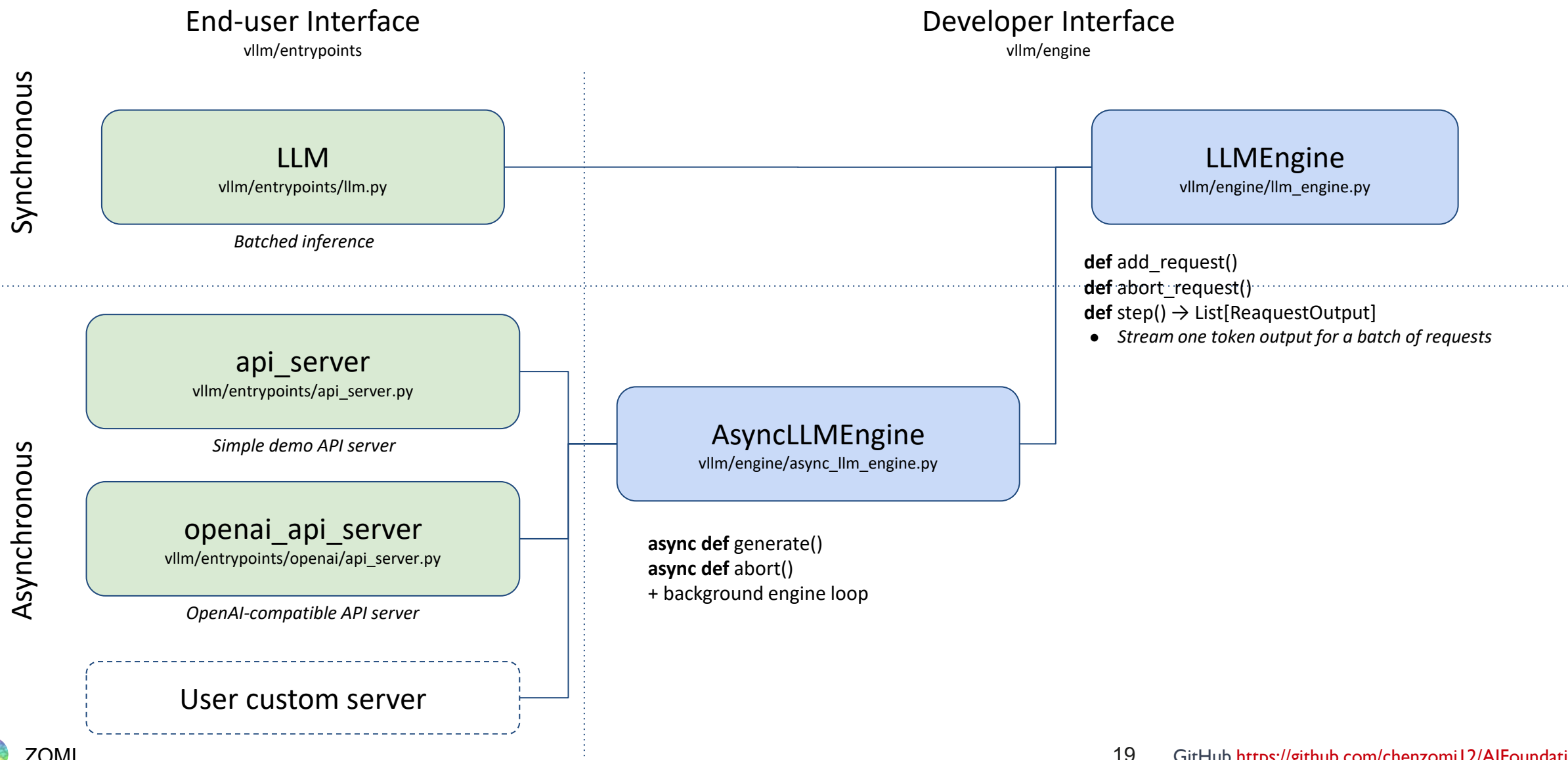
```
$ python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-2-7b-hf
```

Client

```
$ curl http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-2-7b-hf",
    "prompt": "San Francisco is a",
    "max_tokens": 7,
    "temperature": 0
  }'
```



LLMEngine 是 vLLM 的核心逻辑



LLMEngine 是 vLLM 的核心逻辑

- **add_request():**
 - 将每一个请求包装成 vLLM 能处理的数据类型 (Sequence Group) , 并将其加入 Scheduler 的waiting 队列中。该函数按照 “同步” 方式设计, 被设计为 “遍历batch中每条数据, 然后做相应处理” , 只适合批处理场景。
- **abort_request():**
 - 推理过程中, 不是所有请求都能有返回结果。如客户端断开连接时, 该请求推理就可以 abort。
- **step():**
 - 负责执行 1 次推理过程。Scheduler 会决定要送那些数据去执行本次推理, 并负责给这些数据分配好物理块 (metadata) 。模型会根据 metadata, 采用 Paged Attention 完成实际计算。



vLLM 是一个巨大工程

它耦合了调度工程和模型本身的改造

代码中的嵌套非常复杂

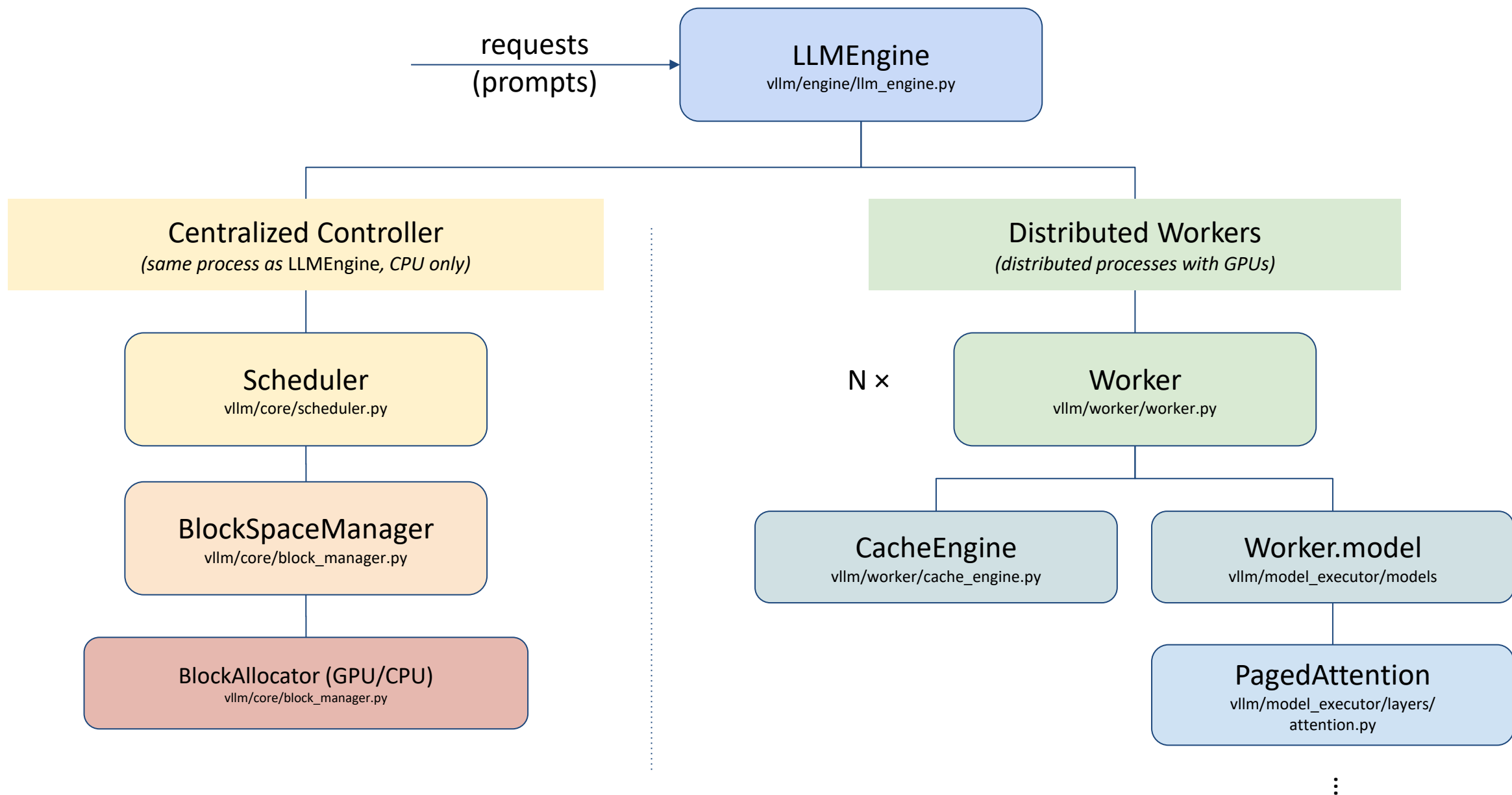


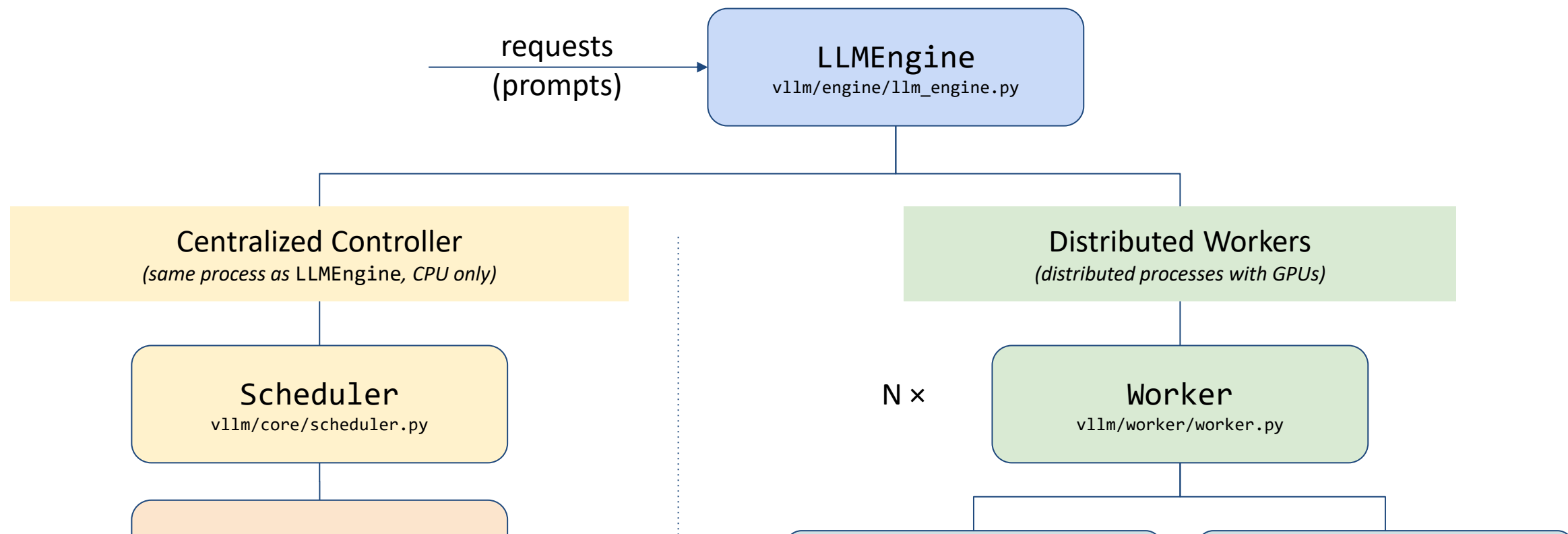
03

VLLM

整体代码架构

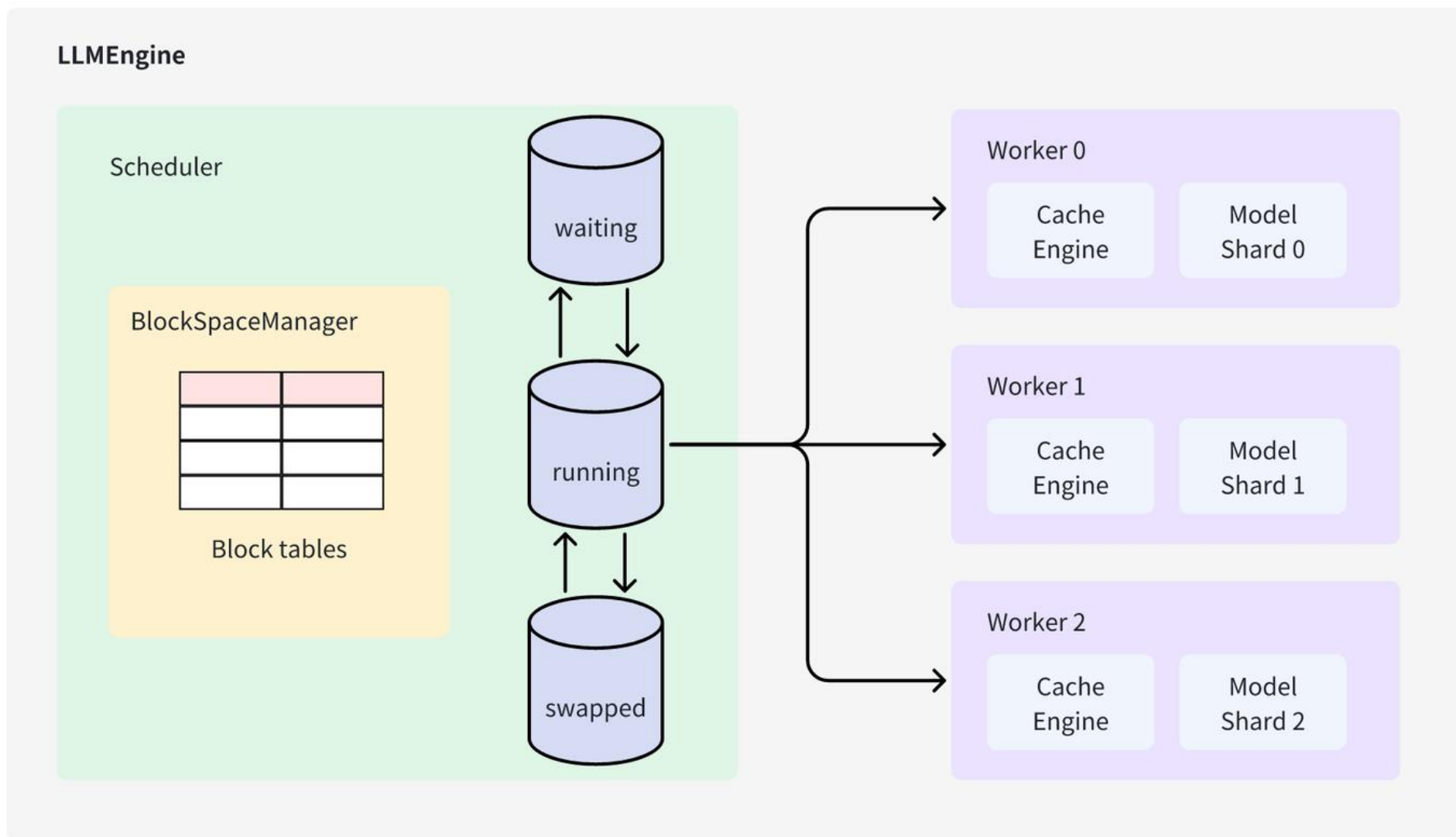


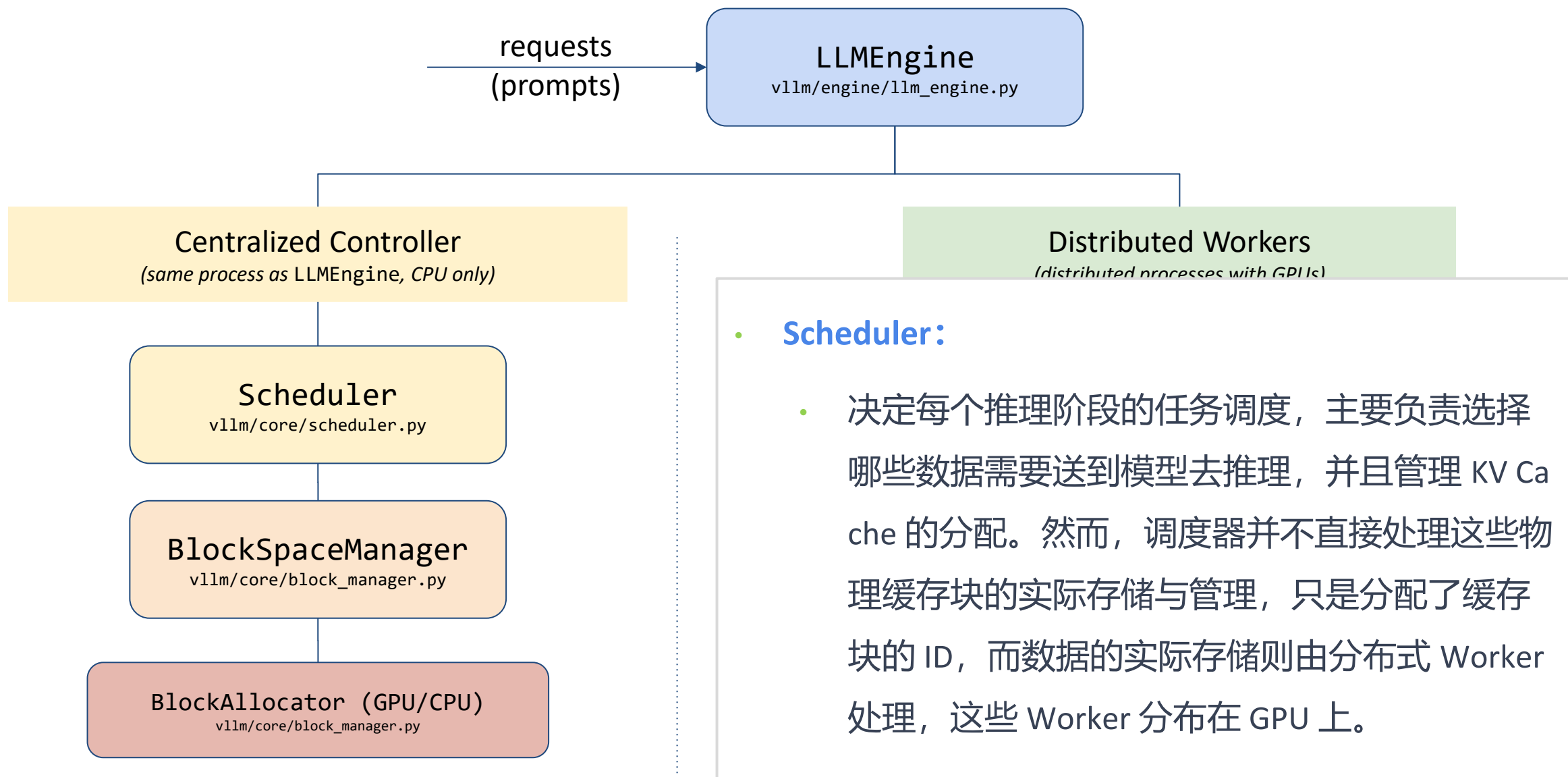




- vLLM 核心组件是 LLMEngine 类，外层接口类 LLM 和 AsyncLLMEngine 都是对 LLMEngine 的封装。
- LLMEngine 有两个核心组件，分别是负责请求调度的 Scheduler 和负责模型推理的 Worker，Scheduler 从等待队列中选择接下来要处理的请求 Worker 负责使用模型对被调度请求进行推理。

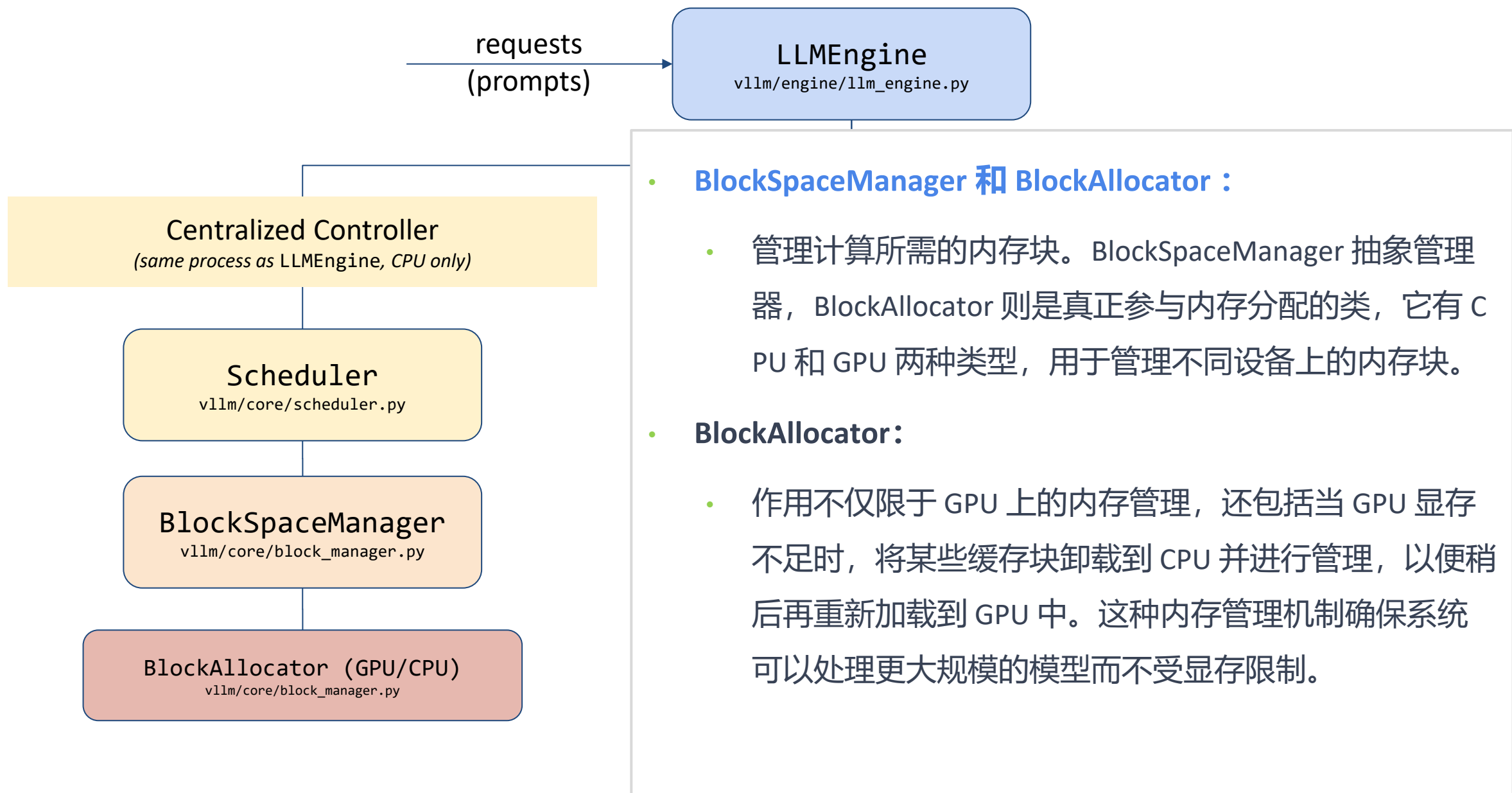
vLLM 整体代码架构

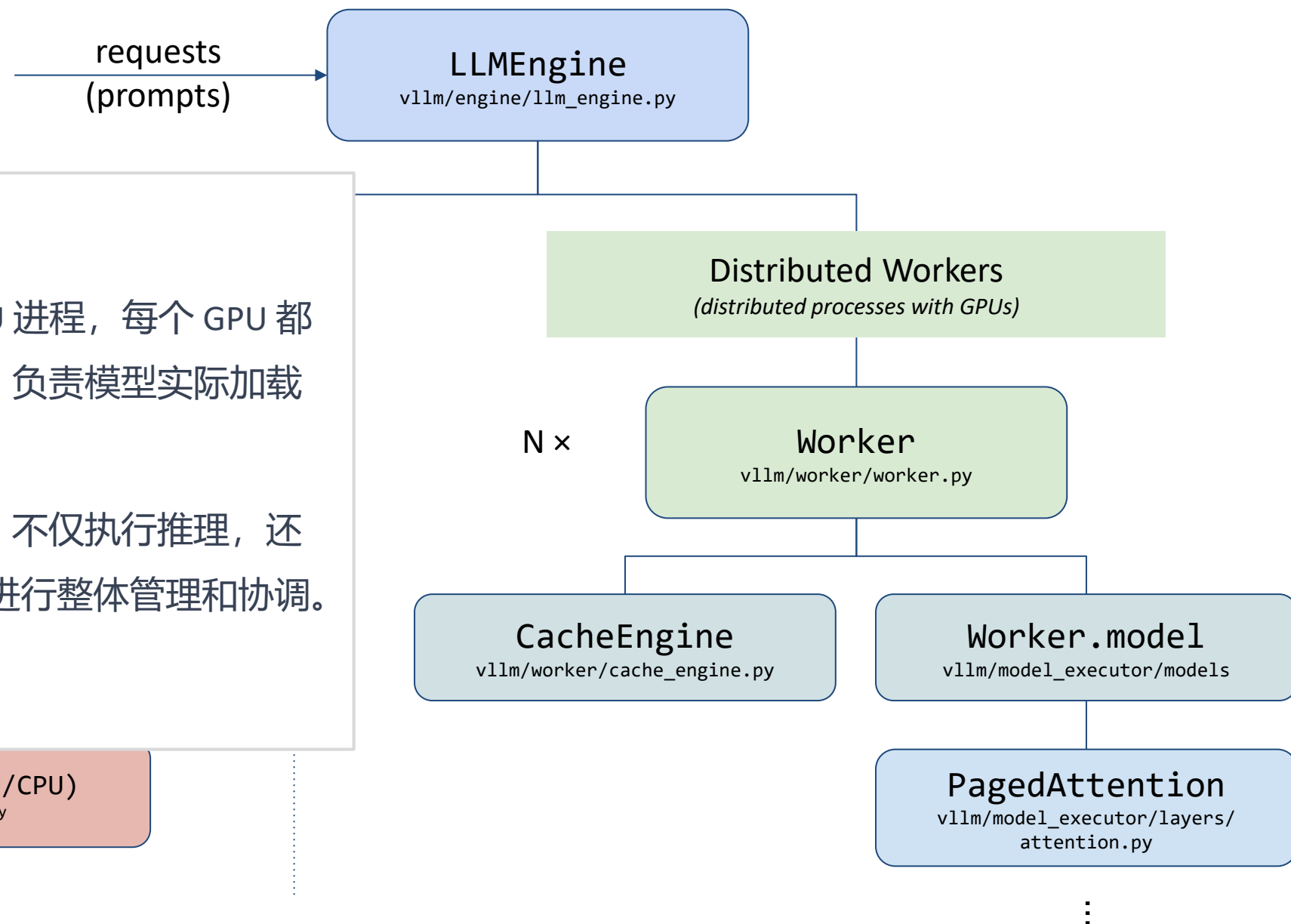




⋮





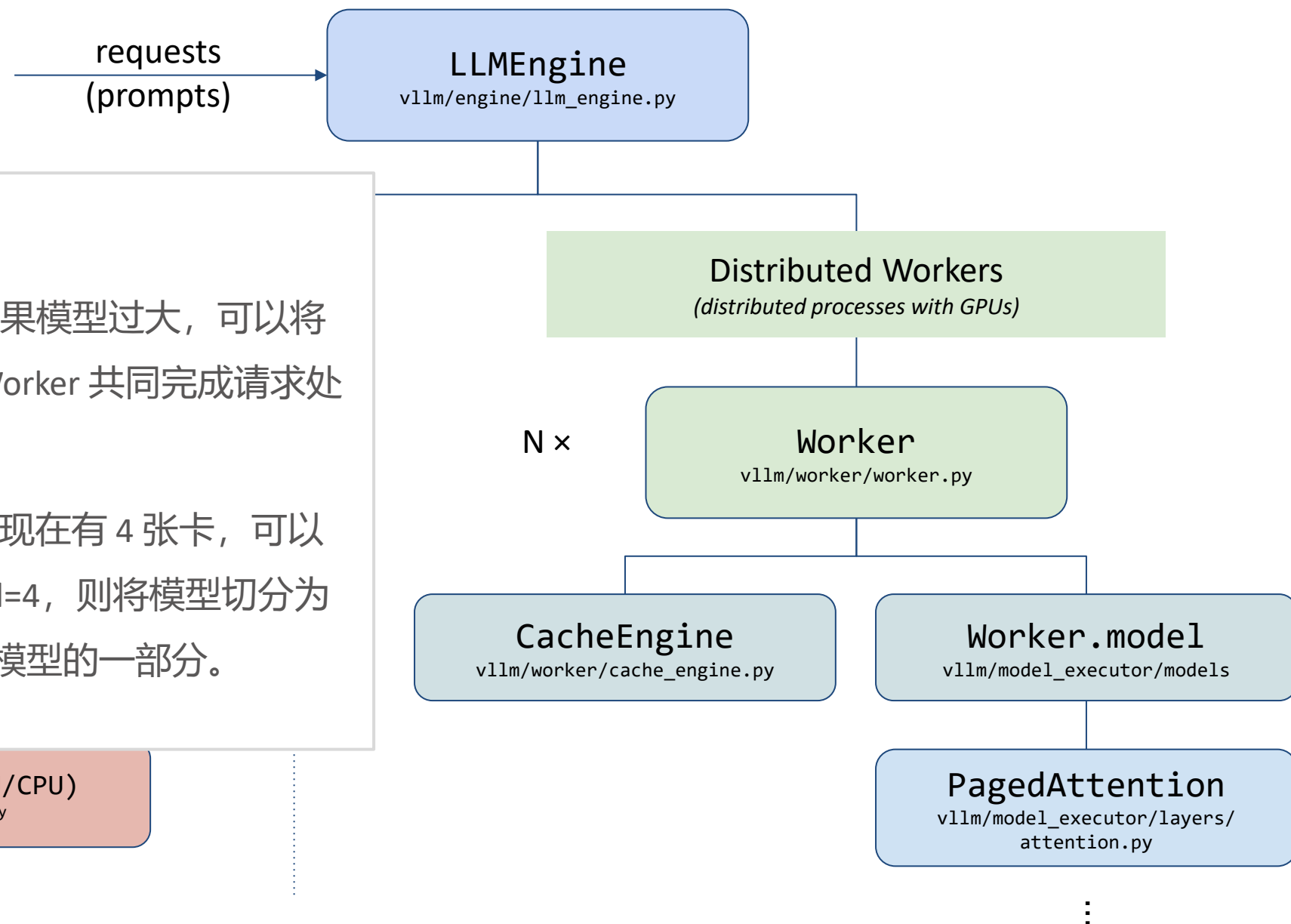


- **Distributed Workers:**

- 可以理解为多个 GPU 进程，每个 GPU 都有一个对应 Worker，负责模型实际加载和推理。
- 架构中称为 Worker，不仅执行推理，还对这些 Worker 实例进行整体管理和协调。

BlockAllocator (GPU/CPU)
vllm/core/block_manager.py





- Worker:**

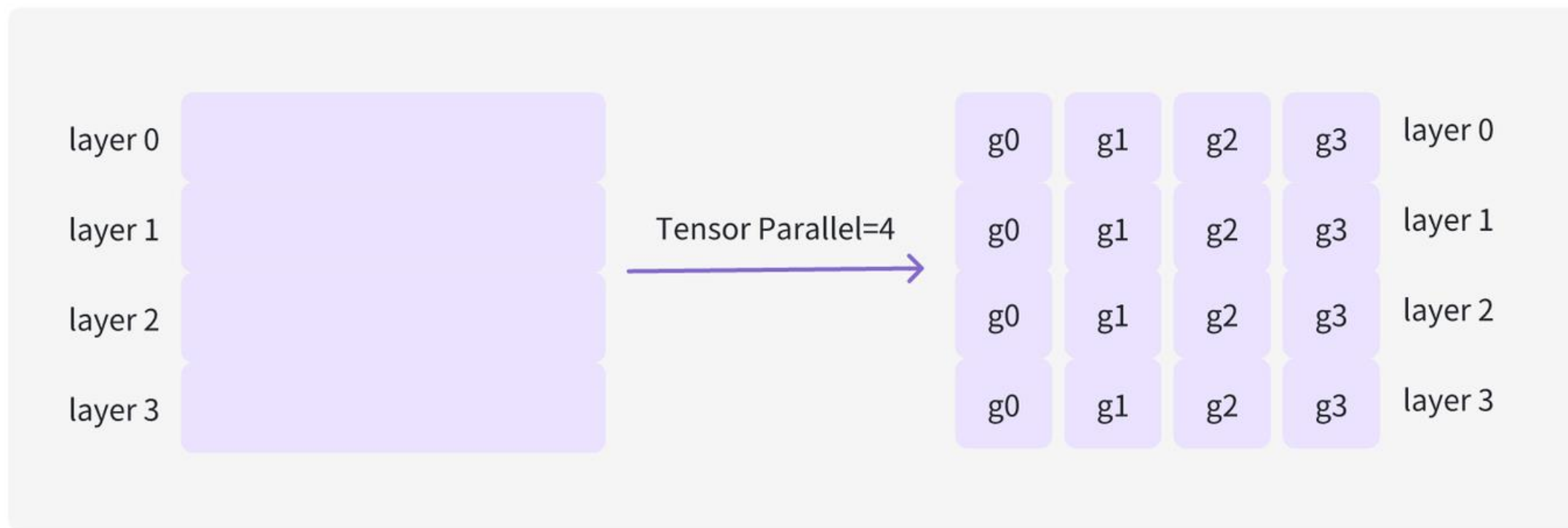
- 负责模型执行。如果模型过大，可以将模型切分到多个 Worker 共同完成请求处理。
- 假设模型有 4 层，现在有 4 张卡，可以设置 Tensor Parallel=4，则将模型切分为 4 份，每张卡存放模型的一部分。

BlockAllocator (GPU/CPU)
`vllm/core/block_manager.py`



CacheEngine

- Worker 一个核心组件是 CacheEngine, 负责 KV cache 初始化以及 KV cache 相关操作。



requests
(prompts)

LLMEngine

vllm/engine/llm_engine.py

Distributed Workers

(distributed processes with GPUs)

N ×

Worker

vllm/worker/worker.py

CacheEngine

vllm/worker/cache_engine.py

Worker.model

vllm/model_executor/models

PagedAttention

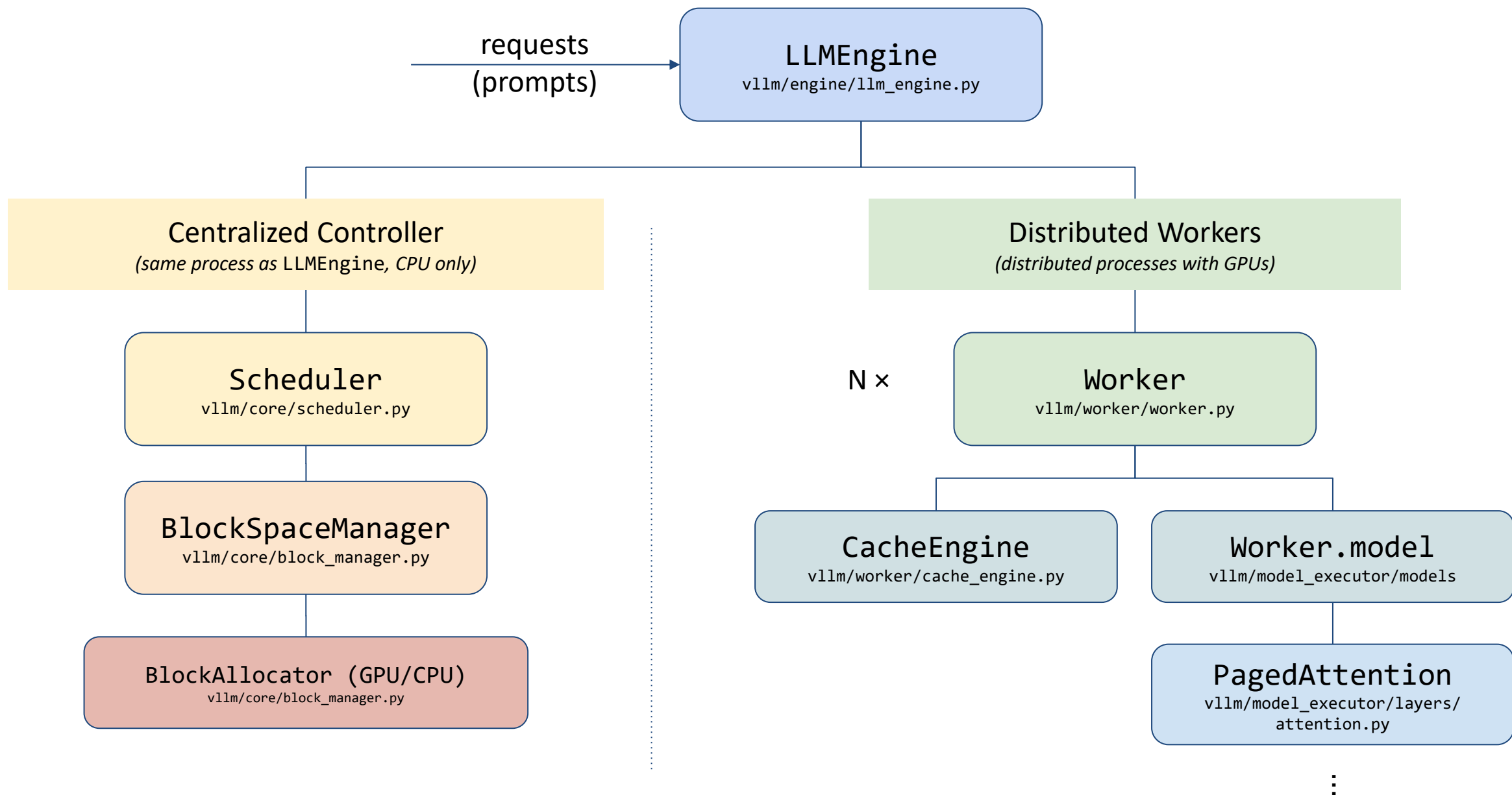
vllm/model_executor/layers/
attention.py

BlockAllocator (GPU/CPU)

vllm/core/block_manager.py

- 每个 Worker 中都有两个关键模块：
- **CacheEngine**：负责管理实际的缓存数据 KV cache，确保所有计算任务都有合适资源可用。
- **Worker.model**：负责模型加载和执行，利用 Paged Attention 组件来高效处理 Attention，从而保证推理任务性能。





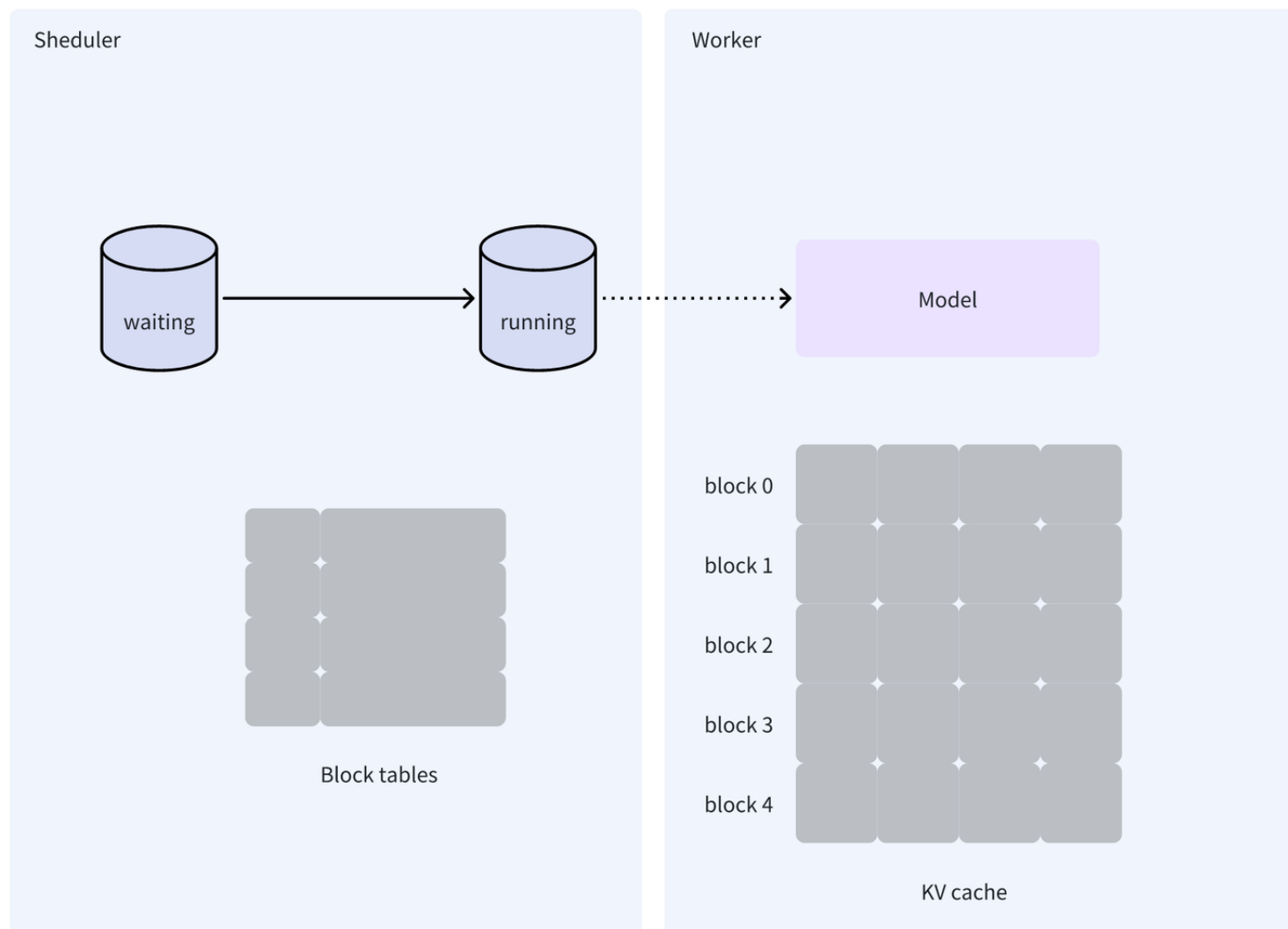
04

Scheduler 和 Worker 的关系



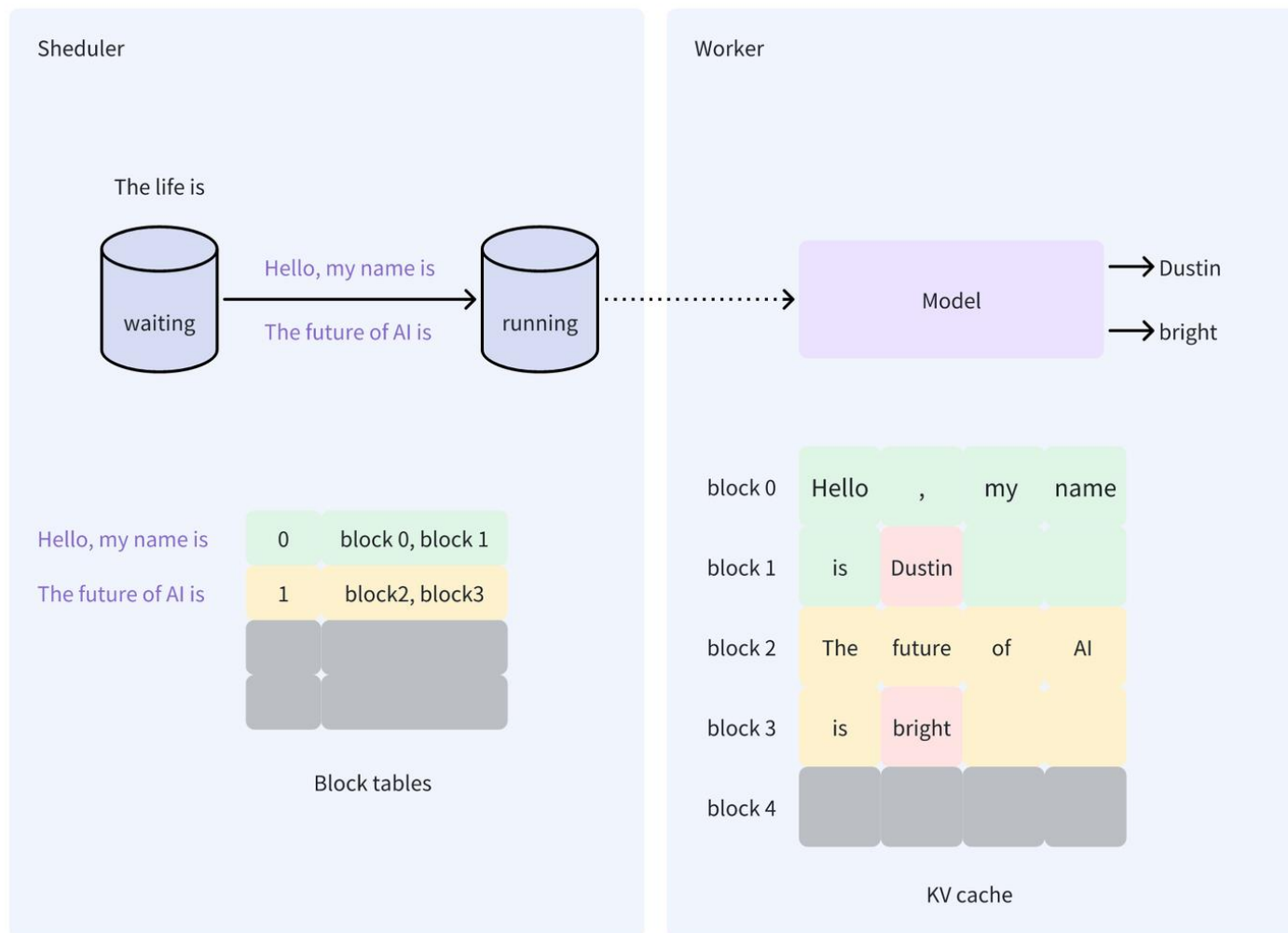
初始化阶段

- 主要初始化 LLMEngine 中 Scheduler 和 Worker 对象
- Scheduler 初始化主要是块表 (block table)
- Worker 初始化包括模型的初始化以及 KV cache 的初始化



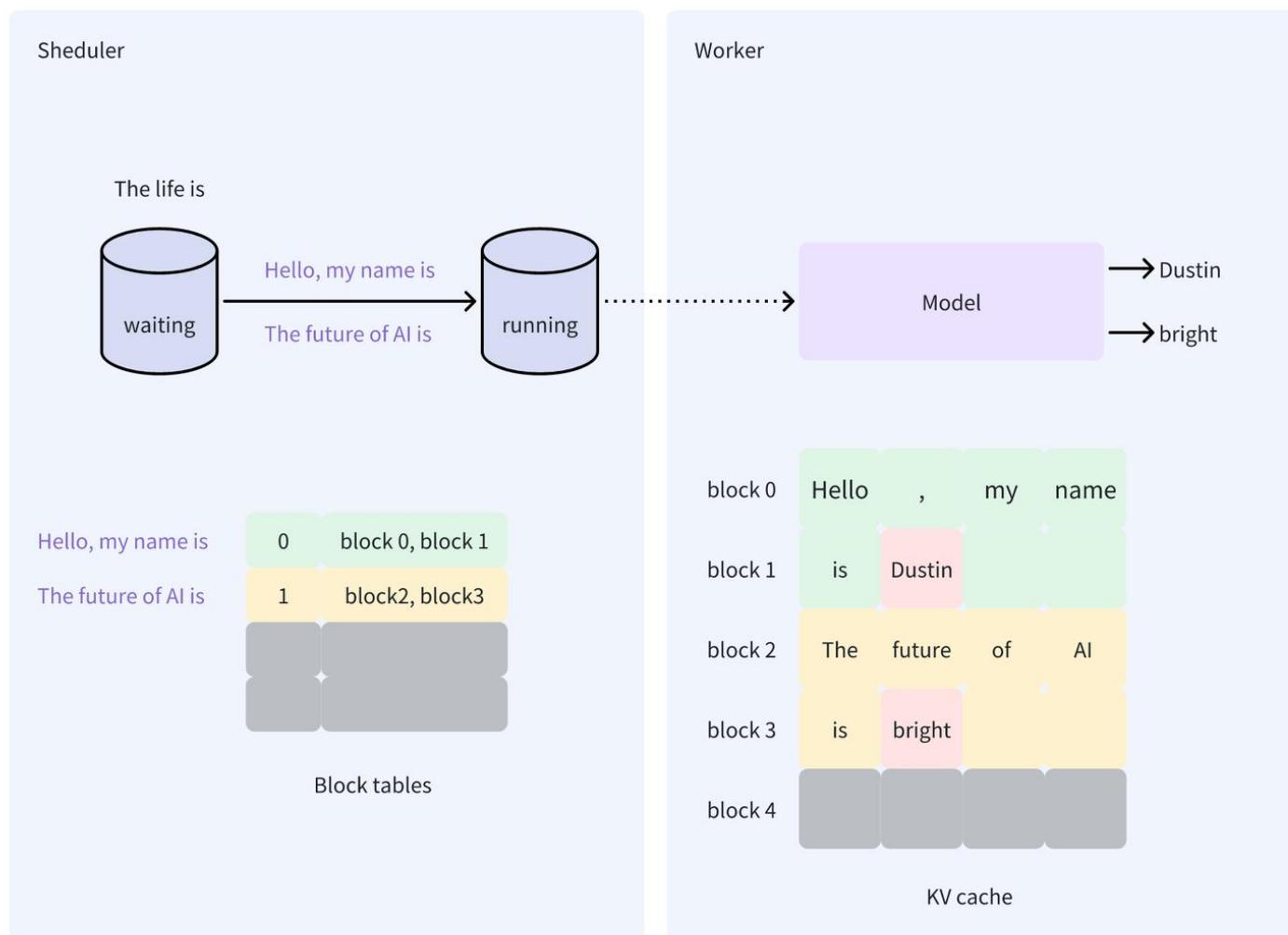
调度和推理

- 设 vLLM 接收到 3 个请求 (s0, s1, s2) 并放入 waiting 队列中
- 它们 prompt 分别为 “Hello, my name is”、“The future of AI is” 和 “The life is”。接下来开始 vLLM 调度和处理



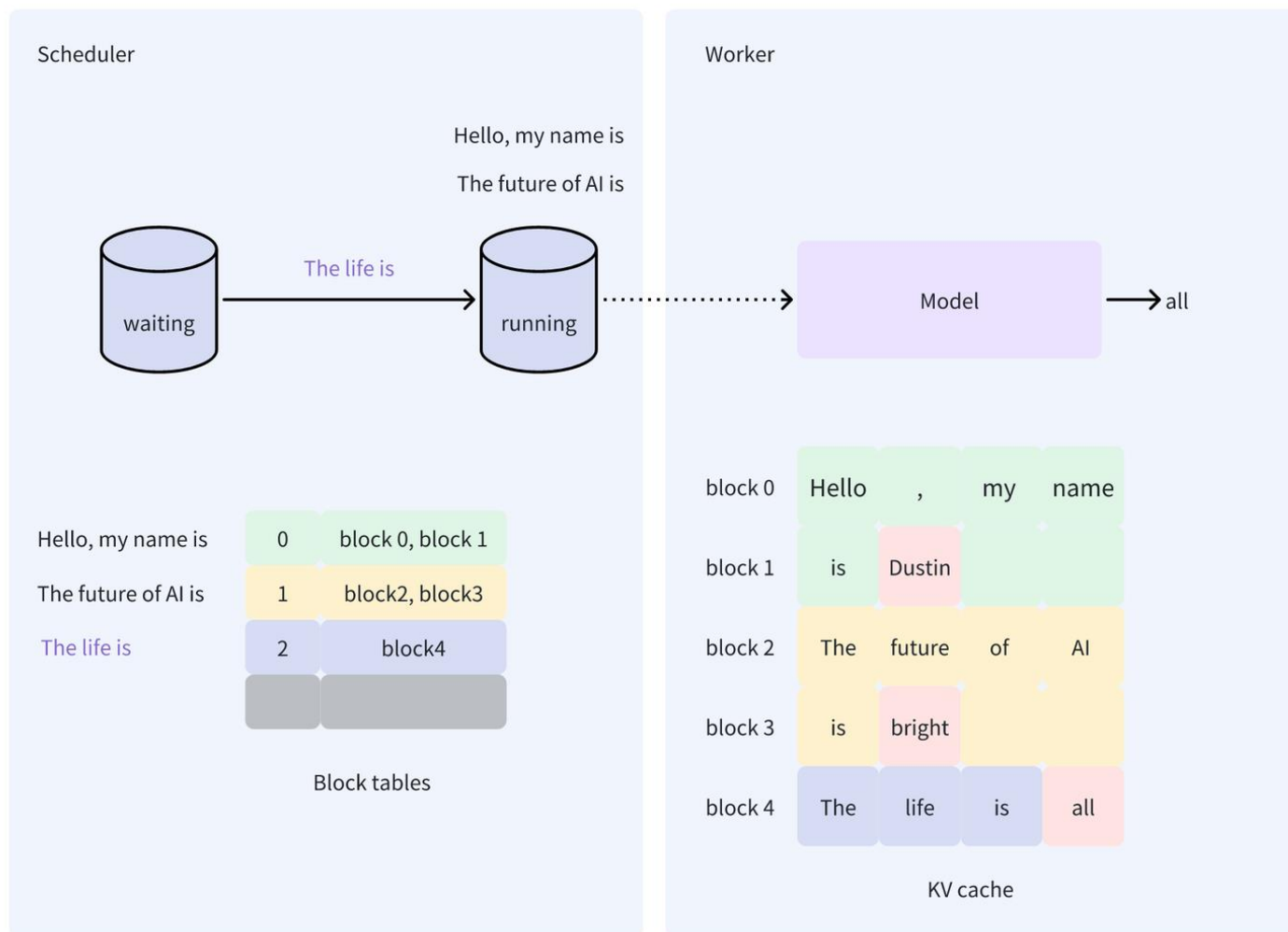
LLM 的第一轮处理

- 设 vLLM 在这一轮只能调度两个请求进行处理，根据 FIFO 原则
- 从 waiting 队列中选择 s0 ("Hello, my name is") 和 s1 ("The future of AI is") 放入到 running 队列
- s0 Worker 生成 token 为 Dustin; s1 Worker 生成 token 为 bright
- 同时，Worker 会将计算过程产生的 KV 值存储在 KV cache 中



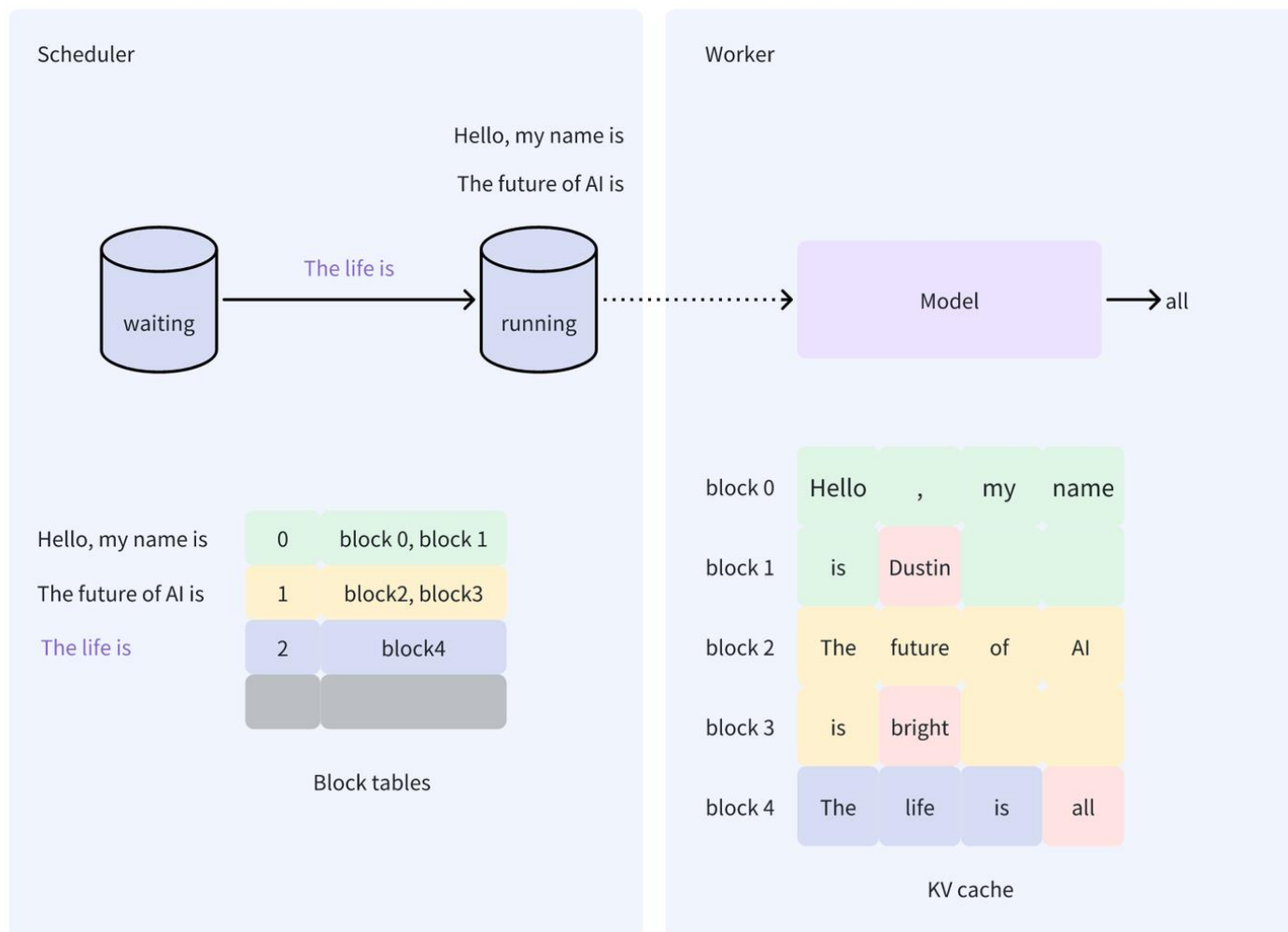
vLLM 的第二轮处理

- 由于 waiting 队列中还有一个请求 s2 (The life is)
- 因此, vLLM 在第二轮只会处理这一个请求, 前面提到, vLLM 只会处理要么都是填充阶段的请求, 要么都是生成阶段的请求。



vLLM 的第三轮处理

- waiting 队列中没有要处理新请求，从 running 队列中选择此轮要处理的请求
- 但由于没有多余空间，vLLM 只会选择 s0 和 s1 进行处理
- 经过多轮调度和推理，最终完成 3 个请求的处理

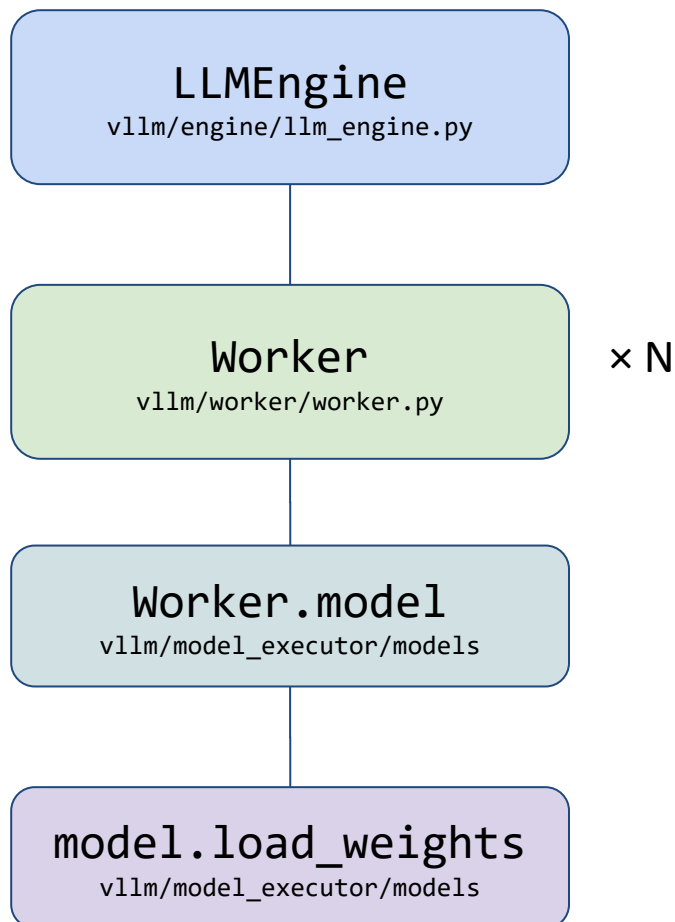


05

vLLM 加载模型 & 预分配显存



Before serving requests *1. Initialize & load model*



- **初始化并加载模型权重**

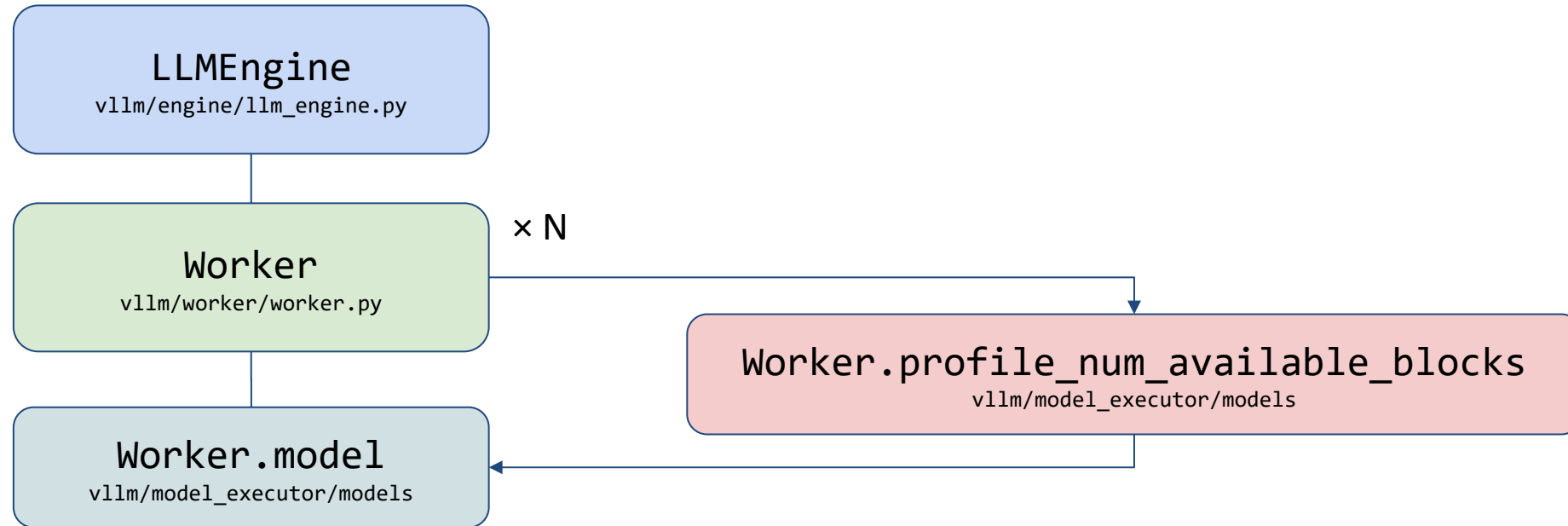
- 具体执行前，vLLM 需要初始化并加载模型权重。
- vLLM 支持从 HF Hub 加载模型，也支持本地加载模型。
- 在加载模型的过程中，vLLM 会将模型权重加载到 GPU 中，以便后续推理任务可以直接在 GPU 上执行。



Initialize & load from
HuggingFace Model Hub



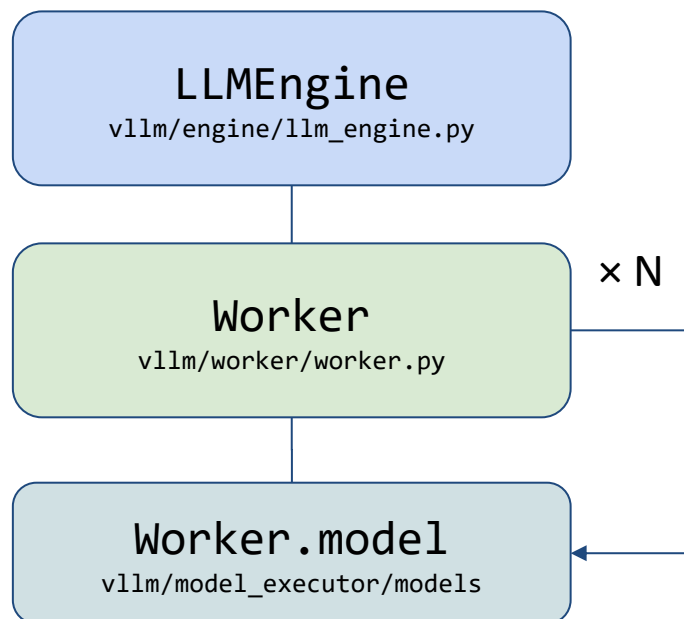
Before serving requests *2. Profile memory usage*



Profile the memory usage with a batch with the max possible #tokens.

`#GPU KV blocks` = remaining memory / block byte size

Before serving requests *2. Profile memory usage*



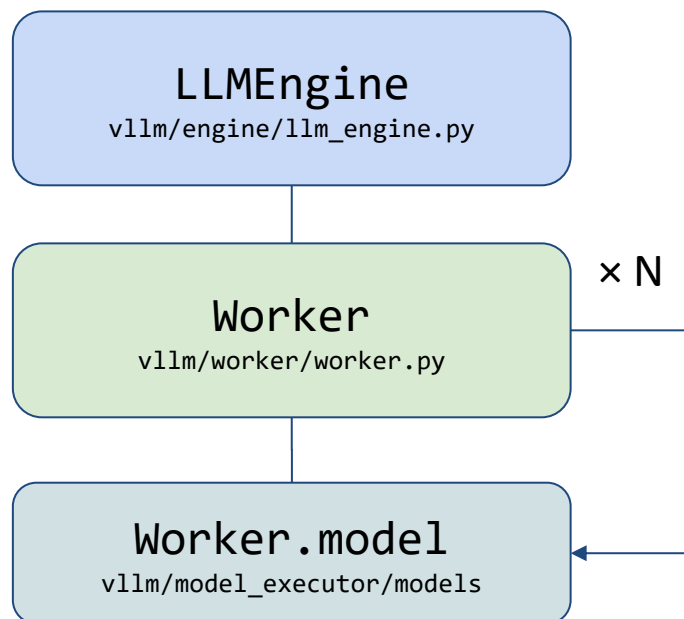
- 通过模拟实验来决定 GPU 和 CPU 上可以分配 KV cache 物理块数量, 确保后续推理时内存分配不会显存溢出 (determine_num_available_blocks)
- 启动时系统会进行“假数据模拟”来测量模型内存使用情况。构造假数据并执行一次模拟前向推理, 观察 GPU 上模型运行时峰值内存需求。
- 前向推理中, 系统不使用 KV cache, 而是单纯地模拟模型推理所需基本内存。帮助确定推理过程会占用多少显存, 为后续内存分配提供依据。



#tokens.

#GPU KV blocks = remaining memory / block byte size

Before serving requests *2. Profile memory usage*



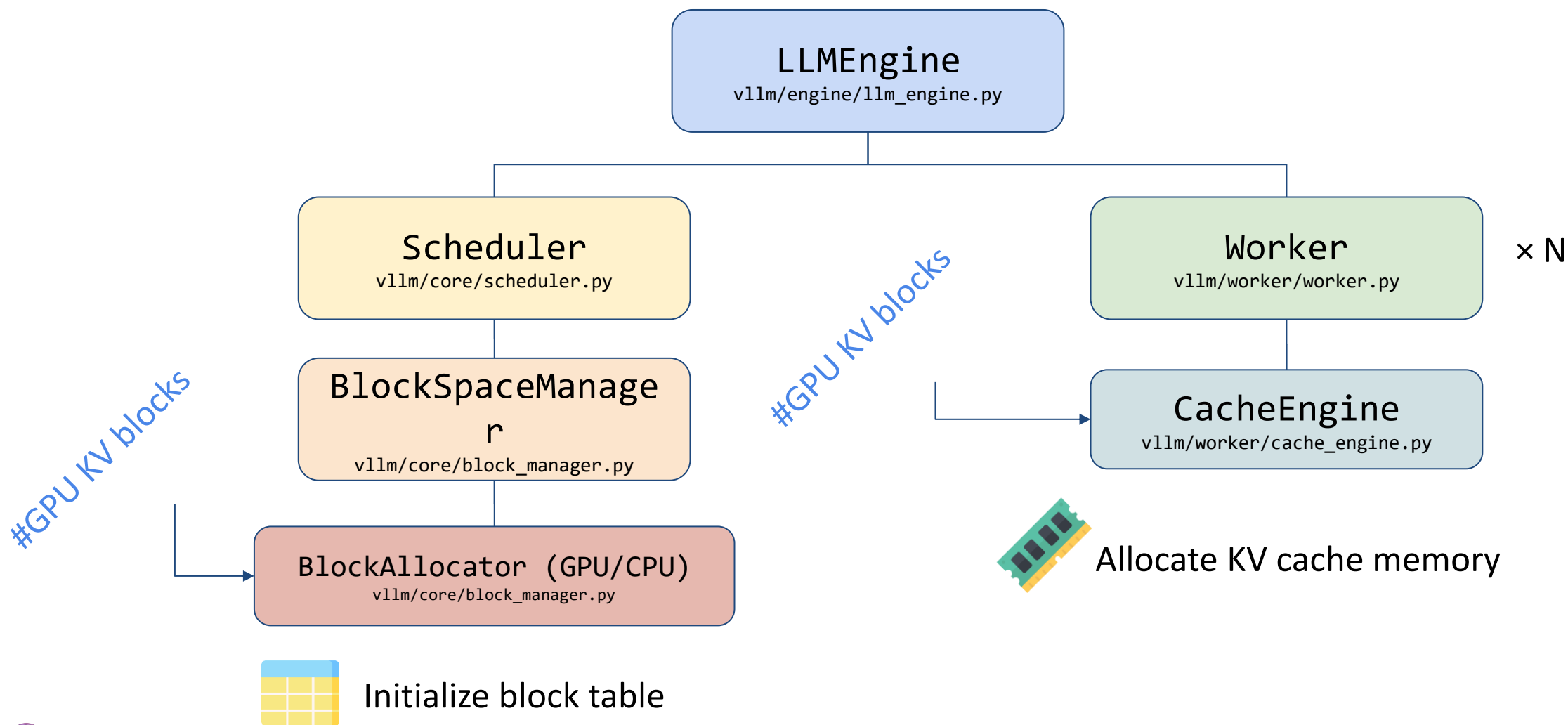
- 完成内存需求测量后，vLLM 使用测得内存数据来计算可分配给 KV cache 显存总量。可以确保显存分配合理，不会因为内存不足而导致 OOM 错误。
- 物理块大小由用户定义，包括多个参数，如 block_size、num_heads、head_size、num_layers 以及数据类型的大小。
- 计算公式会依据这些参数来估算单个物理块的大小，然后根据剩余显存估算出可以分配的物理块总数。



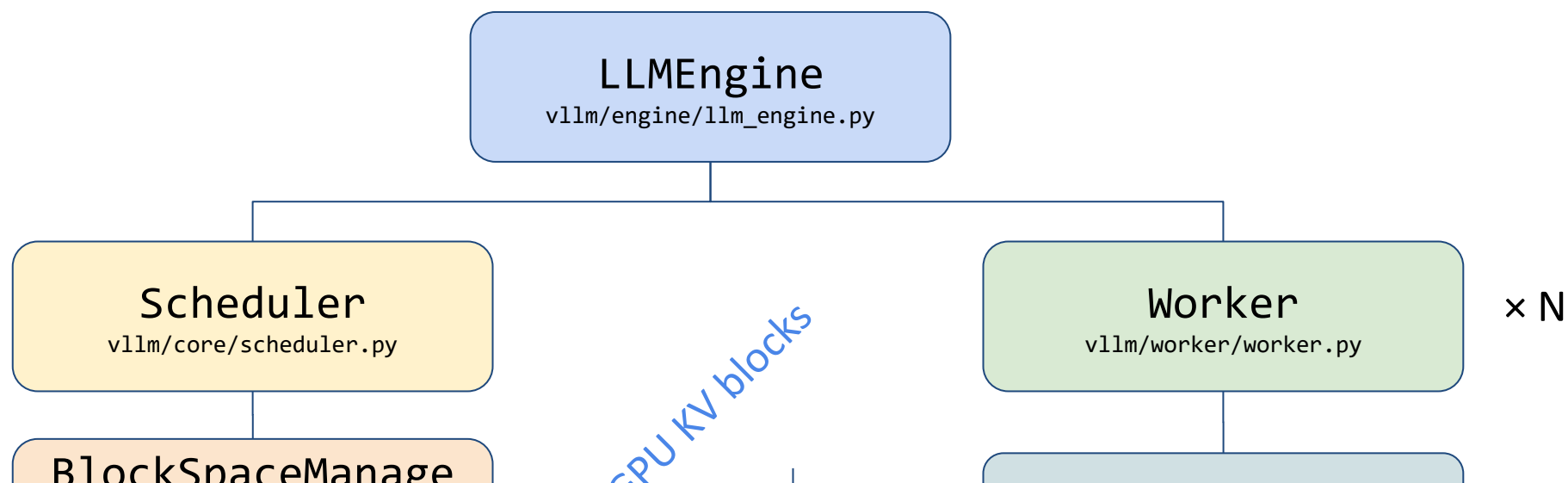
Profile the memory usage with a batch with the max possible #tokens.

#GPU KV blocks = remaining memory / block byte size

Before serving requests 3. Pre-allocate KV Blocks



Before serving requests 3. Pre-allocate KV Blocks



- **预分配 KV Cache**
- 该过程核心是创建 empty tensor，并将它们直接分配到 GPU 上，从而锁定显存用于 KV cache。这种显存预分配方式能够避免推理过程中频繁动态内存分配，提升系统稳定性和推理效率。
- 显存占用比单纯加载模型时更多。因为额外显存已经被预先分配给 KV cache，确保在推理时不会受到其他任务影响。

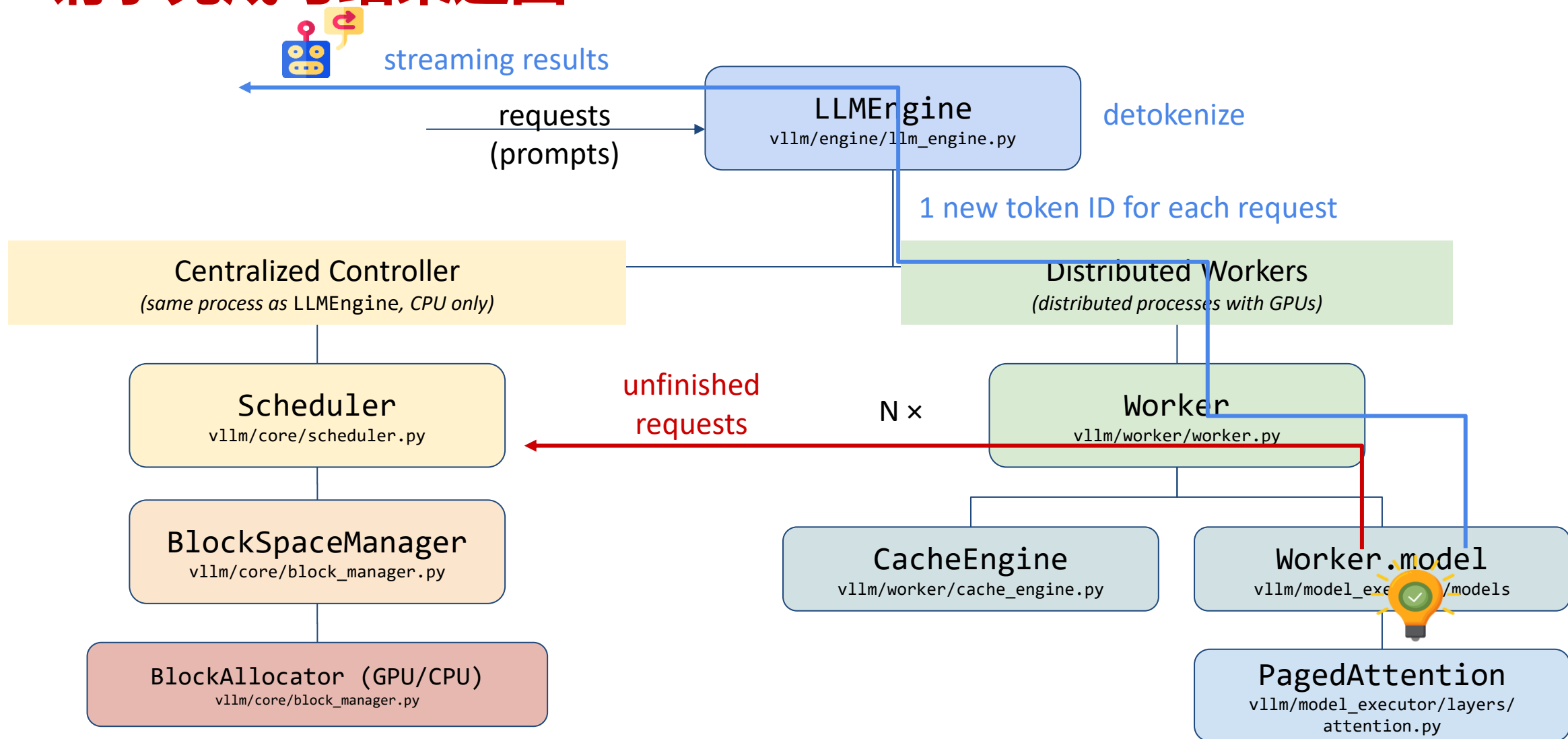
06

vLLM

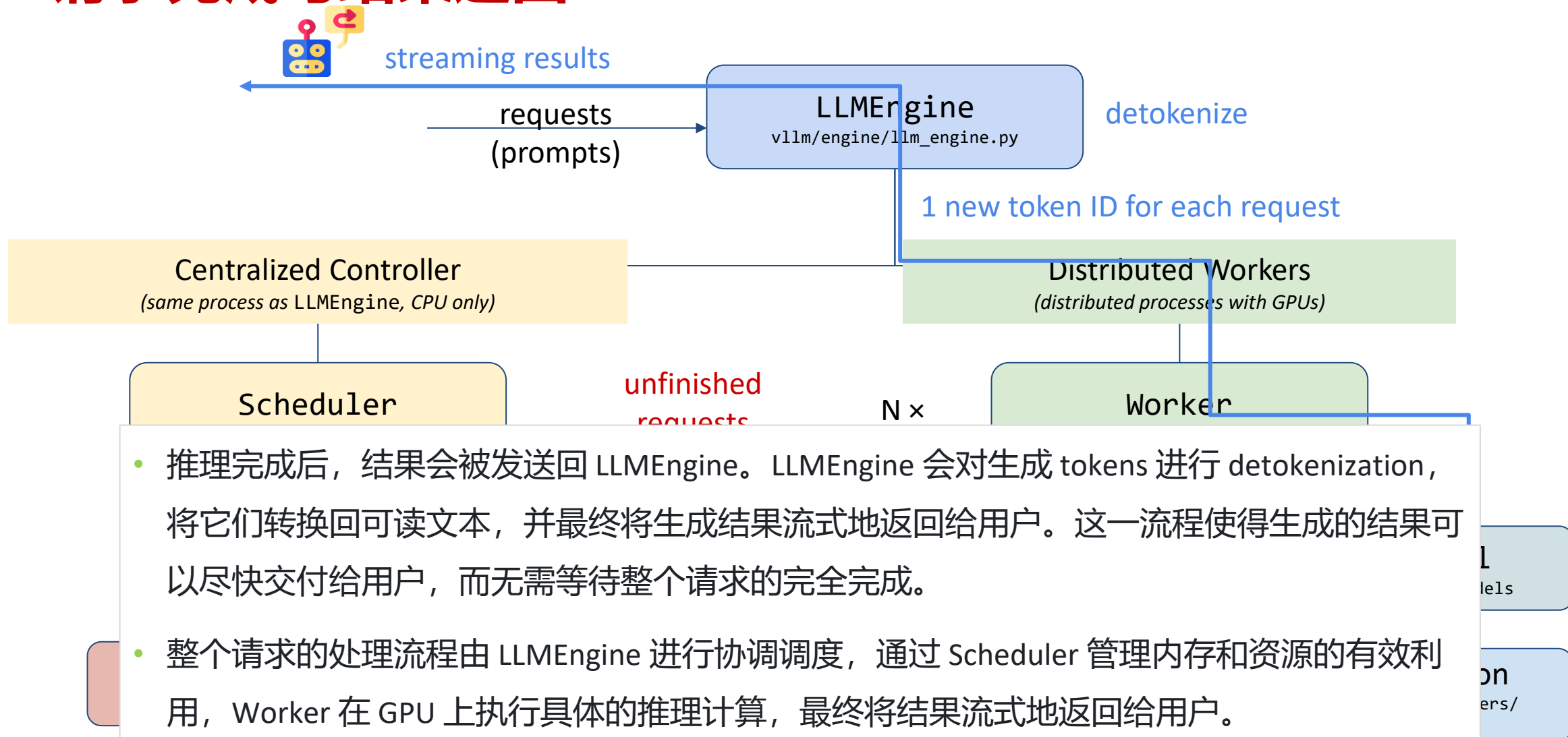
整体架构总结



请求完成与结果返回



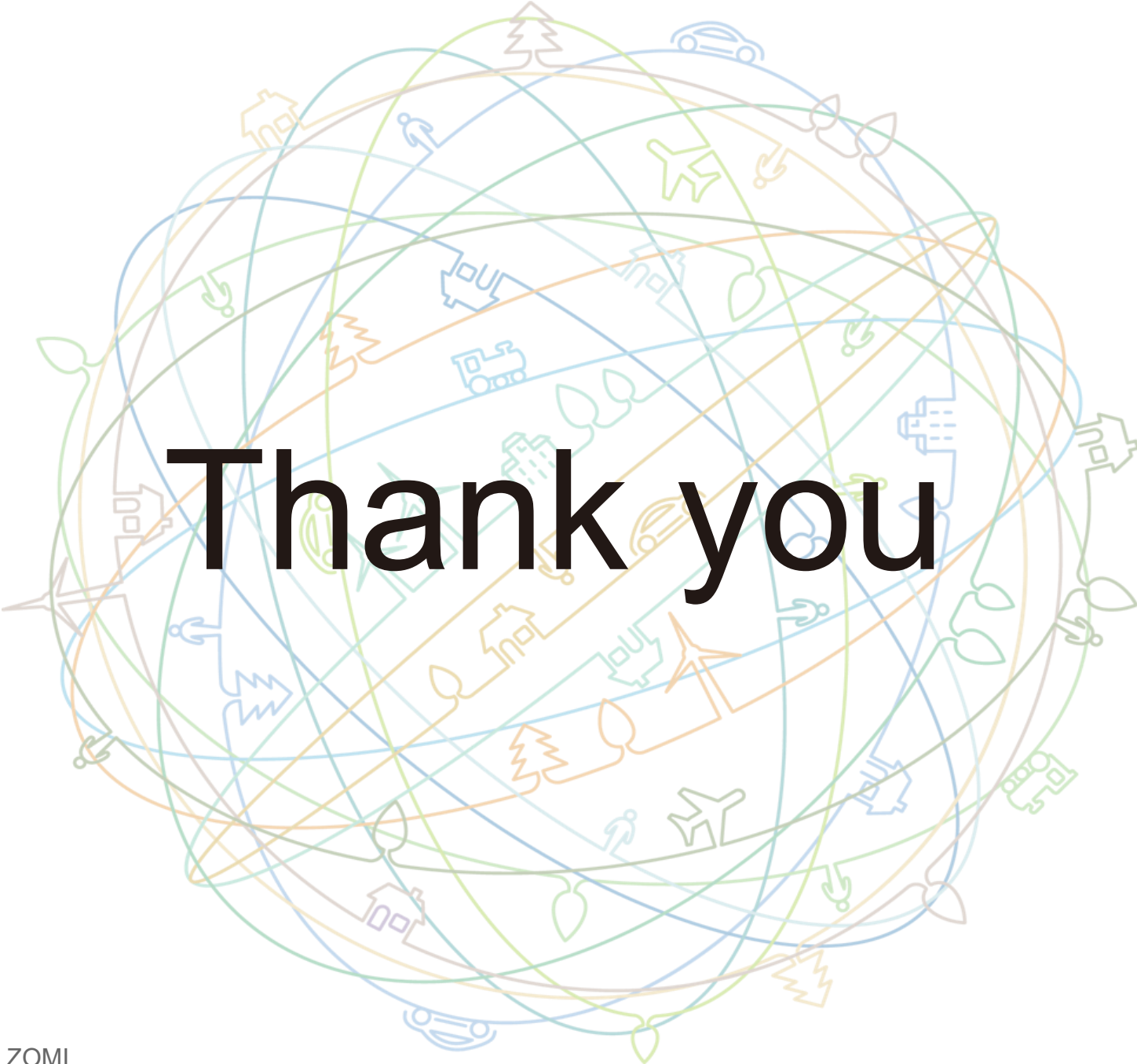
请求完成与结果返回



Summary

1. Core component: **LLMEngine**
2. Centralized controller → Distributed workers
3. Scheduler prepares the requests at each step
4. Workers run the model with **PagedAttention**





Thank you

把AI系统带入每个开发者、每个家庭、
每个组织，构建万物互联的智能世界

Bring AI System to every person, home and
organization for a fully connected,
intelligent world.

Copyright © 2024 XXX Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. XXX may change the information at any time without notice.



GitHub <https://github.com/chenzomi12/AIFoundation>

1. <https://zhuanlan.zhihu.com/p/691045737>
2. https://github.com/PaddleJitLab/CUDATutorial/blob/develop/docs/16_vllm_source_code/01_vllm_arch.md
3. <https://arxiv.org/pdf/2309.06180>
4. https://docs.google.com/presentation/d/1RgUD8aCfcHocghoP3zmXzck9vX3RCI9yfUAB2Bbcl4Y/edit#slide=id.g2716b686c46_0_3

